Université de Rouen Normandie

Master 1 – Génie Informatique et Logiciel

Projet Gestion de Flotte

Rapport de Semaine 7

Observabilité complète · Alertes temps réel · Tests de charge K6 · Déploiement Helm prod-ready

Étudiants :
Ahcène Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Année universitaire :
2025 – 2026

Semaine :
7 / 7

23 avril 2026

Table des matières

1 Observabilité complète **2**
- 1.1 Architecture de la stack . . . 2
- 1.2 Jaeger — Distributed Tracing . . . 2
- 1.3 Prometheus — Métriques . . . 3
- 1.4 Loki + Promtail — Centralisation des logs . . . 3
- 1.5 Alertes métier . . . 3

2 Dashboards Grafana personnalisés **4**
- 2.1 Configuration des datasources . . . 4
- 2.2 Dashboards métier (business metrics) . . . 4

3 Tests end-to-end et tests de charge **5**
- 3.1 Tests E2E — Playwright . . . 5
  - 3.1.1 Organisation des tests . . . 5
  - 3.1.2 Récapitulatif des scénarios . . . 6
- 3.2 Tests de charge — K6 . . . 6
  - 3.2.1 Profils de charge . . . 7
  - 3.2.2 Scripts de test . . . 7

4 Déploiement final avec Helm **8**
- 4.1 Architecture des charts . . . 8
- 4.2 Déploiement automatisé — `scripts/kube.sh` . . . 8
- 4.3 Points d'accès . . . 9

5 Page d'alertes dans le shell frontend **9**
- 5.1 Choix d'intégration . . . 9
- 5.2 Fonctionnalités de `AlertsPage.tsx` . . . 9
  - 5.2.1 Temps réel via subscription GraphQL . . . 9
  - 5.2.2 Filtres et pagination . . . 10
  - 5.2.3 Actions selon le rôle (RBAC) . . . 10
  - 5.2.4 Types d'alertes affichés . . . 11
- 5.3 Intégration dans le Layout et la navigation . . . 11

6 Préparation de la démonstration **11**
- 6.1 Script de simulation d'alertes — `simulate-alerts.sh` . . . 11
- 6.2 Script de simulation GPS — `simulate-GPS.sh` . . . 12
- 6.3 Surveillance Kafka en temps réel — `watch-kafka.sh` . . . 12
- 6.4 Collection Bruno — Tests manuels API . . . 12

7 Bilan de la semaine 7 **14**
- 7.1 Travaux réalisés . . . 14
- 7.2 Points techniques notables . . . 14

1. Observabilité complète

1.1. Architecture de la stack

L'observabilité de la plateforme repose sur le pattern **LGTMo** (Loki, Grafana, Tempo/Jaeger, Métriques via Prometheus, OpenTelemetry), intégralement piloté par le chart Helm `fleet-observability`. L'**OpenTelemetry Collector** constitue le pivot central : il reçoit traces, métriques et logs de tous les microservices via OTLP et les route vers les backends spécialisés.

```yaml
receivers:
  otlp:
    protocols:
      grpc:  { endpoint: "0.0.0.0:4317" }
      http:  { endpoint: "0.0.0.0:4318" }

processors:
  memory_limiter: { limit_percentage: 70, spike_limit_percentage: 30 }
  batch: {}

exporters:
  otlp/jaeger: { endpoint: "jaeger:4317", tls: { insecure: true } }
  loki:        { endpoint: "http://loki:3100/loki/api/v1/push" }
  prometheus:  { endpoint: "0.0.0.0:8889" }

service:
  pipelines:
    traces:  { receivers: [otlp], processors: [batch], exporters: [otlp/jaeger] }
    metrics: { receivers: [otlp], processors: [batch], exporters: [prometheus] }
    logs:    { receivers: [otlp], processors: [batch], exporters: [loki] }
```

Listing 1 – Pipeline de l'OTel Collector — `otel-collector-config.yaml`

1.2. Jaeger — Distributed Tracing

Jaeger (`jaegertracing/all-in-one:1.55`) est déployé en mode *all-in-one* pour la démonstration. Il reçoit les traces via OTLP gRPC (port 4317) depuis l'OTel Collector et expose son interface graphique sous le chemin `/jaeger` de l'ingress `flotte.local`, ce qui permet une navigation unifiée depuis le navigateur.

> **Correction technique appliquée :** La datasource Jaeger dans Grafana doit pointer vers `jaeger:16686/jaeger` (avec le base path) et non `jaeger:16686`. Sans ce correctif, l'exploration de traces depuis les panels Grafana retourne une erreur 404.

1.3. Prometheus — Métriques

Prometheus (`prom/prometheus:v2.51.0`) scrape les métriques agrégées par l'OTel Collector sur le port 8889 avec un intervalle de **5 secondes**, adapté au suivi en quasi-temps réel des événements de la flotte. Les microservices ne s'exposent pas directement à Prometheus : ils poussent leurs métriques en OTLP, ce qui centralise la configuration de collecte.

```yaml
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'otel-collector'
    static_configs:
      - targets: ['otel-collector:8889']
```

Listing 2 – Configuration du scraping Prometheus

1.4. Loki + Promtail — Centralisation des logs

Loki (`grafana/loki:2.9.4`) centralise les logs applicatifs. Promtail est déployé en **DaemonSet** pour collecter les logs au niveau de chaque nœud Kubernetes. Cette approche complète l'OTel Collector pour les composants qui n'émettent pas nativement en OTLP (frontend React, `location-service` Node.js).

> **Problème rencontré sous Minikube :** Promtail ne trouvait pas les fichiers de log à cause de symlinks Docker absents. Solution appliquée : configuration `static_configs` pointant directement vers `/var/log/pods/**/*.log` au lieu des symlinks `/var/log/containers`.

1.5. Alertes métier

Les alertes métier sont produites par le `events-service` à partir de la consommation des topics Kafka. La chaîne complète est la suivante :

1. Un événement arrive sur un topic Kafka métier (`location.events`, `alertes.events` ou `conducteurs.events`).
2. Le `events-service` consomme l'événement, crée une alerte en base et publie `ALERT_CREATED` sur `flotte.alerts.created`.
3. La Gateway GraphQL propage l'alerte via la subscription WebSocket `ALERT_CREATED_SUB` vers tous les clients connectés.
4. Le frontend met à jour la vue `/alerts` sans rechargement.

Type d'alerte	Topic source	Sévérité
`SPEED_EXCEEDED`	`flotte.location.events`	WARNING / HIGH
`GEOFENCING_BREACH`	`flotte.location.events`	HIGH
`VEHICLE_IMMOBILIZED`	`flotte.location.events`	WARNING
`MAINTENANCE_OVERDUE`	`flotte.alertes.events`	CRITICAL / HIGH
`LICENSE_EXPIRING`	`flotte.conducteurs.events`	WARNING
`LICENSE_EXPIRED`	`flotte.conducteurs.events`	CRITICAL

Table 1 – Types d'alertes métier et topics Kafka associés

2. Dashboards Grafana personnalisés

2.1. Configuration des datasources

Grafana 10.4.1 est configuré avec trois datasources déclarées en tant que *provisioned resources* (fichiers YAML montés dans le pod), ce qui évite toute configuration manuelle après déploiement.

Datasource	URL interne	Usage
Prometheus (défaut)	`http://prometheus:9090`	Métriques techniques et métier
Loki	`http://loki:3100`	Logs applicatifs
Jaeger	`http://jaeger:16686/jaeger`	Traces distribuées

Table 2 – Datasources Grafana provisionnées automatiquement

2.2. Dashboards métier (business metrics)

Les dashboards couvrent quatre domaines fonctionnels :

— **Vue flotte** : nombre de véhicules disponibles, en maintenance, en tournée, hors service. Évolution temporelle du statut du parc.

— **Vue conducteurs** : conducteurs actifs, en tournée, indisponibles. Alertes permis expirant sous 30 jours.

— **Vue alertes** : répartition des alertes actives par sévérité (CRITICAL, HIGH, WARNING, INFO). Taux d'acquittement.

— **Vue performance API** : latences p50/p95/p99 des endpoints GraphQL, REST et gRPC. Taux d'erreur par service.

Grafana est accessible sous `http://flotte.local/grafana` (chemin relatif configuré via `GF_SERVER_ROOT_URL` et `GF_SERVER_SERVE_FROM_SUB_PATH=true`).

3. Tests end-to-end et tests de charge

3.1. Tests E2E — Playwright

Les tests end-to-end sont implémentés avec **Playwright** au niveau du Monorepo (`frontend/playwright.config.ts`). Trois profils de rôles sont couverts : administrateur, manager, technicien. Chaque profil possède son propre fichier d'authentification sauvegardé (`e2e/.auth/{user,manager,technicien}.json`), ce qui évite une reconnexion Keycloak à chaque scénario.

3.1.1. Organisation des tests

```
frontend/e2e/tests/
  auth.setup.ts              # Login admin        -> .auth/user.json
  auth.setup.manager.ts      # Login manager      -> .auth/manager.
      json
  auth.setup.technicien.ts   # Login technicien   -> .auth/technicien.
      json
  dashboard.spec.ts          # Statistiques tableau de bord
  vehicles.spec.ts           # CRUD vehicules
  drivers.spec.ts            # CRUD conducteurs
  maintenance.spec.ts        # Suivi des interventions
  location.spec.ts           # Carte GPS temps reel
  roles.spec.ts              # Controle d'acces par role (RBAC)
```

Listing 3 – Arborescence des tests E2E

3.1.2. Récapitulatif des scénarios

Module	Scénario testé	Résultat
Auth (admin)	Connexion Keycloak + sauvegarde session	Pass
Auth (manager)	Connexion avec rôle manager	Pass
Auth (technicien)	Connexion avec rôle technicien	Pass
Dashboard	Présence des 4 cartes de statistiques	Pass
Dashboard	Valeurs chargées sans erreur GraphQL	Pass
Véhicules	Liste, création, suppression	Pass
Conducteurs	Liste et absence d'erreur GraphQL	Pass
Conducteurs	Création (UUID v4, email unique)	Pass
Conducteurs	Modification (mise à jour téléphone)	Pass
Conducteurs	Changement de statut (`ON_LEAVE`)	Pass
Conducteurs	Suppression (soft delete vérifié)	Pass
Maintenance	Liste, planification, modification statut	Pass
Localisation	Chargement carte Leaflet	Pass
RBAC	Technicien redirigé depuis `/drivers`	Pass
RBAC	Manager redirigé depuis `/maintenance`	Pass

Table 3 – Récapitulatif des 15 scénarios end-to-end Playwright — 0 échec (Minikube)

```
./scripts/test-e2e.sh docker     # cible http://localhost:3005
./scripts/test-e2e.sh minikube   # cible http://flotte.local
```

Listing 4 – Lancement des tests E2E

3.2. Tests de charge — K6

Les tests de charge sont implémentés avec **K6** et couvrent les trois protocoles utilisés par la plateforme : GraphQL (HTTP/JSON), REST et gRPC (streaming).

3.2.1. Profils de charge

Profil	VUs max	Durée	Seuil d'acceptation
Smoke	1	1 min	p(99) < 1 500 ms
Load	10	4 min	p(95) < 500 ms
Stress	100	4 min	taux d'erreur < 1 %

TABLE 4 – Profils de charge K6 définis dans `scenarios/options.js`

Le profil *load* suit un *ramp-up* progressif : 0 → 10 VUs en 1 minute, maintien 2 minutes, descente en 1 minute, afin d'observer le comportement de la plateforme sous charge réaliste.

3.2.2. Scripts de test

```
load-tests/
  utils/config.js                # Auth Keycloak + helpers
  scenarios/options.js            # Profils smoke / load / stress
  targets/
    graphql-vehicles.js           # POST /graphql -> liste vehicules
    rest-alerts.js                # GET  /api/alerts -> liste alertes
    grpc-location.js              # gRPC WatchVehicle (server-
        streaming)
```

Listing 5 – Arborescence des scripts K6

- **`graphql-vehicles.js`** : requête GraphQL `GetVehicles` avec jeton Bearer. Vérifie le statut HTTP 200, l'absence d'erreurs GraphQL et la présence du tableau `vehicles`.
- **`rest-alerts.js`** : appel REST `GET /api/alerts` avec authentification. Vérifie le statut 200 et que la réponse est un tableau.
- **`grpc-location.js`** : ouvre un flux `LocationService/WatchVehicle` en gRPC server-streaming, teste la connexion et le half-close propre.

```
./scripts/load-tests.sh docker     # smoke -> load -> stress (Docker
    Compose)
./scripts/load-tests.sh minikube   # smoke -> load -> stress (Minikube
    )
```

Listing 6 – Lancement des tests K6

4. Déploiement final avec Helm

4.1. Architecture des charts

L'ensemble du système est déployé via trois charts Helm indépendants, permettant de mettre à jour chaque couche séparément :

```
infra/helm/
  fleet-infra/          # PostgreSQL (TimescaleDB), Kafka (KRaft), Redis
  fleet-observability/  # Loki, Grafana, Jaeger, Prometheus, OTel, Promtail
  fleet-app/            # 8 services applicatifs + frontend + ingress
```

Listing 7 – Structure des charts Helm

- **`fleet-infra`** : infrastructure stateful. Gère les bases de données PostgreSQL/TimescaleDB par service, Kafka en mode KRaft (sans ZooKeeper) et Redis pour le cache.
- **`fleet-observability`** : stack d'observabilité. Toutes les ressources Kubernetes (Deployments, Services, ConfigMaps) sont générées depuis les templates du chart.
- **`fleet-app`** : couche applicative. Déploie les 6 microservices métier, la Gateway GraphQL, le frontend React, Keycloak et les règles d'ingress.

4.2. Déploiement automatisé — `scripts/kube.sh`

Le script `kube.sh` orchestre l'intégralité du déploiement Minikube en une seule commande :

```
# 1. Demarrage Minikube + addon ingress
minikube start --cpus=4 --memory=8192
minikube addons enable ingress

# 2. Namespace et secrets
kubectl create namespace flotte-namespace
kubectl create secret generic db-secrets ...

# 3. Build des images dans le daemon Docker de Minikube
eval $(minikube docker-env)
docker build -t vehicle-service:latest ./services/vehicle-service
# ... (driver, maintenance, events, location, gateway, frontend)

# 4. Installation des charts dans l'ordre
helm install fleet-infra      ./infra/helm/fleet-infra
helm install fleet-observability ./infra/helm/fleet-observability

# 5. Creation des topics Kafka
kubectl exec -n flotte-namespace kafka-0 -- kafka-topics.sh \
  --create --topic flotte.location.events ...

```

```
# 6. Deploiement applicatif
helm install fleet-app ./infra/helm/fleet-app
```

Listing 8 – Étapes du script `kube.sh`

4.3. Points d'accès

Service	URL
Frontend shell	`http://flotte.local`
GraphQL Gateway	`http://flotte.local/graphql`
Keycloak	`http://flotte.local/auth`
Grafana	`http://flotte.local/grafana`
Jaeger UI	`http://flotte.local/jaeger`

Table 5 – Points d'accès de la plateforme sous Minikube

5. Page d'alertes dans le shell frontend

5.1. Choix d'intégration

La page `/alerts` a été intégrée **directement dans l'application shell** (`shell/src/alerts/AlertsPage.tsx`) plutôt que comme micro-frontend dédié. Ce choix se justifie par deux raisons :

1. **Taille fonctionnelle** : la fonctionnalité tient en un seul composant React ; créer un remote Module Federation pour cela aurait ajouté de la complexité de build sans valeur ajoutée.
2. **Transversalité** : les alertes concernent tous les rôles et doivent être accessibles depuis n'importe quel contexte de navigation ; les placer dans le shell garantit leur disponibilité permanente.

5.2. Fonctionnalités de `AlertsPage.tsx`

5.2.1. Temps réel via subscription GraphQL

Les nouvelles alertes apparaissent instantanément sans rechargement grâce à une subscription WebSocket `ALERT_CREATED_SUB` via Apollo Client.

```
const ALERT_CREATED_SUB = gql`
  subscription OnAlertCreated {
    alertCreated {
      id
      type
      severity
      status
```

```
      message
      vehicleId
      driverId
      createdAt
    }
  }
`;

useSubscription(ALERT_CREATED_SUB, {
  onData: ({ data }) => {
    const newAlert = data.data?.alertCreated;
    if (newAlert) {
      setAlerts(prev => [newAlert, ...prev]);
    }
  },
});
```

Listing 9 – Subscription temps réel des alertes — `AlertsPage.tsx`

5.2.2. Filtres et pagination

— Filtre par **statut** : ACTIVE, ACKNOWLEDGED, RESOLVED, EXPIRED

— Filtre par **sévérité** : CRITICAL, HIGH, WARNING, INFO

— Pagination : 20 alertes par page

5.2.3. Actions selon le rôle (RBAC)

Action	admin	manager	technicien
Acquitter (*acknowledge*)	✓	✓	×
Résoudre (*resolve*)	✓	✓	✓

Table 6 – Contrôle d'accès aux actions sur les alertes

5.2.4. Types d'alertes affichés

Code	Label affiché
GEOFENCING_BREACH	Sortie de zone
SPEED_EXCEEDED	Excès de vitesse
VEHICLE_IMMOBILIZED	Immobilisation véhicule
MAINTENANCE_OVERDUE	Maintenance en retard
LICENSE_EXPIRING	Permis expirant
LICENSE_EXPIRED	Permis expiré
DELIVERY_TOUR_DELAYED	Tournée en retard

Table 7 – Types d'alertes métier et leurs libellés dans l'interface

5.3. Intégration dans le Layout et la navigation

La route `/alerts` est déclarée dans l'application Shell et ajoutée au menu de navigation du `Layout.tsx`. Elle est visible par tous les rôles (admin, manager, technicien).

```
import AlertsPage from './alerts/AlertsPage';

<Route path="alerts" element={<AlertsPage />} />
```

Listing 10 – Route /alerts dans `App.tsx` (shell)

6. Préparation de la démonstration

6.1. Script de simulation d'alertes — `simulate-alerts.sh`

Le script `scripts/simulate-alerts.sh` génère des scénarios réalistes en s'appuyant sur les véhicules et conducteurs réels présents en base (récupérés via une requête GraphQL). Il publie ensuite les événements directement dans Kafka.

```
./scripts/simulate-alerts.sh minikube  # ou docker
```

Listing 11 – Lancement de la simulation

Type d'événement	Topic Kafka	Détail
SPEED_EXCEEDED	`flotte.location.events`	138, 108, 65 km/h
GEOFENCING_BREACH	`flotte.location.events`	Zone Nord, Périmètre Rouen
VEHICLE_IMMOBILIZED	`flotte.location.events`	Véhicule arrêté trop longtemps
MAINTENANCE_OVERD	`flotte.alertes.events`	Sévérité CRITICAL et HIGH
LICENSE_EXPIRING	`flotte.conducteurs.events`	5 et 12 jours restants
LICENSE_EXPIRED	`flotte.conducteurs.events`	Permis expiré

Table 8 – Événements générés par le script de simulation

6.2. Script de simulation GPS — `simulate-GPS.sh`

Le script `scripts/simulate-GPS.sh` simule un véhicule en mouvement en envoyant des coordonnées GPS réelles via gRPC directement vers le `location-service`. Le trajet part de Rouen et progresse en temps réel ; la carte Leaflet du module `location` se met à jour automatiquement via la subscription WebSocket.

```
./scripts/simulate-GPS.sh minikube  # ouvre le port-forward gRPC si besoin
./scripts/simulate-GPS.sh docker
```

Listing 12 – Lancement de la simulation GPS

6.3. Surveillance Kafka en temps réel — `watch-kafka.sh`

Pendant la démonstration, le script `scripts/watch-kafka.sh` permet d'afficher en direct les événements circulant sur tous les topics `flotte.*`. Il supporte les deux environnements :

```
./scripts/watch-kafka.sh docker     # via docker exec sur flotte-kafka
./scripts/watch-kafka.sh minikube   # via kubectl exec sur le pod Kafka
```

Listing 13 – Surveillance Kafka en temps réel

Cela permet de montrer le flux événementiel complet : de la publication GPS jusqu'à l'apparition de l'alerte dans le navigateur.

6.4. Collection Bruno — Tests manuels API

La collection Bruno (`bruno/`) permet de rejouer manuellement n'importe quel appel API pendant la démonstration :

Catégorie	Nb requêtes	Services couverts
GraphQL	34	Véhicules, conducteurs, maintenance, alertes, localisation
REST	37	CRUD complet de tous les microservices
gRPC	2	WatchVehicle, StreamPositions

Table 9 – Collection Bruno : 73 requêtes API prêtes pour la démonstration

L'authentification Keycloak est gérée automatiquement par un *pre-request script* Bruno : il échange les credentials contre un JWT, détecte l'expiration (marge de 120 secondes) et injecte le token dans l'en-tête `Authorization` de chaque requête. Deux environnements sont pré-configurés : `docker` et `minikube`.

7. Bilan de la semaine 7

7.1. Travaux réalisés

Fonctionnalité	Statut
Stack observabilité complète (Jaeger, Prometheus, Loki, Grafana)	**Terminé**
Pipeline OTel Collector (traces + métriques + logs)	**Terminé**
Dashboards Grafana (business metrics)	**Terminé**
Page `/alerts` dans le shell (temps réel, RBAC, filtres)	**Terminé**
Tests E2E Playwright (3 rôles, RBAC, 15 scénarios)	**Terminé**
Tests de charge K6 (smoke / load / stress, GraphQL + REST + gRPC)	**Terminé**
Déploiement Helm prod-ready (3 charts, script kube.sh)	**Terminé**
Scripts de simulation (alertes + GPS)	**Terminé**
Collection Bruno (73 requêtes)	**Terminé**
Intégration CI/CD (frontend + location dans le pipeline)	**Terminé**

Table 10 – Bilan des délivrables de la semaine 7

7.2. Points techniques notables

— **Pas de micro-frontend dédié pour les alertes** : le composant est intégré directement dans le shell. Ce choix évite la surcharge architecturale d'un remote Module Federation pour une fonctionnalité simple et transversale.

— **K6 couvre les trois protocoles** : la plupart des projets ne testent que le REST ; ici le gRPC streaming est également testé en charge, ce qui est représentatif de l'usage réel du `location-service`.

— **Alertes entièrement événementielles** : aucun polling ; le chemin complet Kafka → events-service → GraphQL subscription → frontend est temps réel de bout en bout.